Documentación Técnica Integral del Código — Sistema Zona P (NexaFlow)

> **Sistema de Automatización de Flujo Laboral y Gestión de Turnos**
> **Empresa:** Parqueaderos Zona P (Medellín, Colombia)
> **Stack:** React 19 + TypeScript + Tailwind CSS + Vite 8 + SQLite (`node:sqlite`) + n8n Webhooks

1. Arquitectura General del Sistema

El sistema está diseñado bajo una arquitectura modular desacoplada donde el frontend en React interactúa de forma reactiva con un motor de cálculo laboral colombiano, un contexto de estado centralizado, y persiste sus datos en una base de datos local SQLite mediante un middleware HTTP integrado.

2. Inventario de Estructura de Archivos

```
nexaflow/
├── database/
│   ├── schema.sql           # DDL: 9 tablas relacionales, índices y vistas
│   ├── seed.sql             # Datos iniciales: sedes, 34 operarios, 12 puestos, 41
horarios
│   ├── init_sqlite.js       # Script que compila schema y seed en database/zonap.db
│   └── zonap.db             # Archivo binario de la base de datos SQLite
├── server/
│   ├── sqliteService.js     # Capa DAO: consultas y mutaciones sobre SQLite
│   ├── apiRouter.js         # Router HTTP para atender /api/*
│   └── index.js             # Servidor HTTP autónomo en Node.js (puerto 3001)
├── src/
│   ├── types/
│   │   └── index.ts         # Definiciones TypeScript de entidades y contratos
│   ├── utils/
│   │   ├── colombianLaborEngine.ts # Motor normativo 2026 (42h, noche 19-06, festivos)
│   │   └── payrollExportService.ts # Exportador a Excel (.xlsx), CSV y TXT
│   ├── services/
│   │   ├── apiService.ts    # Cliente fetch hacia /api/*
│   │   └── n8nIntegrationService.ts # Webhooks n8n, WhatsApp y Habeas Data
│   ├── context/
│   │   └── AppContext.tsx   # Estado global reactivo, sincronizador y toasts
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Header.tsx   # Barra superior, filtros, roles y estado de BD
│   │   │   └── Sidebar.tsx  # Barra de navegación lateral de 5 módulos
│   │   ├── schedule/
│   │   │   ├── ScheduleView.tsx    # Grilla matricial de Puestos x Días
│   │   │   ├── AssignSlotModal.tsx # Modal de asignación de celda
│   │   │   ├── PuestoModal.tsx     # Creación/edición de puestos y horarios
│   │   │   ├── NoveltyModal.tsx    # Modal de novedad operativa (RF-03)
│   │   │   └── SendScheduleModal.tsx # Disparo de WhatsApp vía n8n
│   │   ├── employees/
│   │   │   └── EmployeesView.tsx   # Directorio, altas, bajas, roles y colores
│   │   ├── reports/
│   │   │   └── PayrollReportsView.tsx # Consolidado de nómina y exportaciones
│   │   ├── audit/
│   │   │   └── AuditLogView.tsx    # Bitácora inmutable de novedades
│   │   ├── admin/
│   │   │   └── AdminConfigView.tsx # Configuración legal 2026 y backups
│   │   └── common/
│   │       └── ToastContainer.tsx  # Alertas flotantes en tiempo real
│   ├── App.tsx              # Componente raíz y switch de módulos
```

```
│   └── main.tsx              # Punto de entrada de React en el DOM
├── vite.config.ts            # Configuración Vite con middleware SQLite
└── package.json              # Dependencias y scripts de ejecución
```

3. Base de Datos SQLite (`database/`)

3.1

Define la estructura relacional estricta optimizada para SQLite:

- **Tabla `sedes`**: Almacena las ubicaciones físicas (`Sede 1 Centro`, `Sede 2 Norte`, `Sede 3 Laureles`), colores corporativos y estado activo.
- **Tabla `empleados`**: Directorio de colaboradores. Contiene identificación (`cedula`), nombre, cargo, rol de sistema con constraint `CHECK(rol_sistema IN ('admin', 'coordinador', 'operario', 'auditor_rrhh'))`, teléfono (para WhatsApp), email y los colores hexadecimales de la celda en la malla (`color_badge_bg`, `color_badge_text`, `color_badge_border`).
- **Tabla `puestos`**: Puestos operativos configurables por el coordinador (`puesto # 1`, `Rondero`, `Puesto # 12 Privado`).
- **Tabla `horarios_puesto`**: Franjas horarias asociadas a cada puesto con clave foránea `REFERENCES puestos(id) ON DELETE CASCADE`. Almacena horas en formato `'HH:MM'` y días activos en JSON (`'[1,2,3,4,5]'`).
- **Tabla `asignaciones_turno`**: Matriz operativa central. Contiene `UNIQUE(puesto_id, horario_id, fecha_turno)` para garantizar que nunca existan dos asignaciones conflictivas en la misma celda. La clave foránea a `empleados` usa `ON DELETE SET NULL` para que si se borra un empleado, el turno quede vacante sin corromper la grilla.
- **Tabla `bitacora_novedades` (RF-03)**: Registro inmutable de auditoría. Guarda quién modificó el turno, motivo tipificado (`Calamidad doméstica`, `Inclemencias del clima`, `Fuerza mayor`, `Ausentismo`, `Otro`), y la serialización JSON del estado previo y nuevo. Usa `ON DELETE RESTRICT` para evitar borrar registros de auditoría legal.
- **Tabla `acumulados_laborales_2026` (RF-01 / RF-02)**: Totalización semanal por empleado de horas ordinarias, recargos nocturnos, extras y flags de sobrecupo (42h).
- **Tabla `notificaciones_n8n_logs`**: Trazabilidad de envíos a n8n y WhatsApp con reintentos y HTTP status.
- **Tabla `copias_seguridad`**: Registro de snapshots automáticos con política de retención de 90 días.
- **Vistas SQL**:

- `v_matriz_turnos_completa`: Une puestos, horarios, fechas y datos del empleado en una sola consulta plana.
- `v_reporte_nomina_consolidado`: Agrupa y totaliza horas para previsualizar nómina.

3.2

Puebla la base de datos con los datos exactos tomados de la operación real de Zona P:

- 3 Sedes operativas.
- 34 Empleados reales con la paleta de colores de la imagen (pasteles celestes, verdes, fucsias, amarillos, naranjas y grises).
- 12 Puestos de trabajo (`puesto # 1`, `Puesto # 2`, `Puesto # 4`, `Puesto # 5`, `Puesto # 6`, `Puesto # 7`, `puesto # 8`, `Puesto # 9`, `Puesto # 10`, `Rondero`, `Puesto # 12 Privado`, `puesto # 11`).
- 41 Franjas horarias operativas distribuidas en mañana, tarde, noche y partidos.
- Asignaciones de ejemplo para la semana del 26 de octubre al 01 de noviembre de 2026.

3.3

Script de ejecución automática con `node database/init_sqlite.js`:

- Usa el módulo estándar `DatabaseSync` de Node.js v22+.
- Habilita `PRAGMA foreign_keys = ON;`.
- Lee secuencialmente `schema.sql` y `seed.sql`, ejecutando el DDL y los inserts en una única transacción atómica.
- Imprime un conteo verificado de registros por tabla.

4. Capa Backend & Acceso a Datos (`server/`)

4.1

Capa de acceso a datos (Data Access Object) que aísla las consultas SQL de la capa HTTP:

- **`getDatabase()`**: Retorna una instancia activa de `DatabaseSync` con verificación de claves foráneas.
- **`getDbHealth()`**: Realiza un conteo rápido (`COUNT(*)`) de todas las tablas principales y valida el estado de conexión del archivo `database/zonap.db`.
- **`getAllData()`**: Consulta maestros (`sedes`, `empleados`, `puestos`, `horarios`, `asignaciones`, `bitacora`) y los mapea a los formatos de tipos de TypeScript esperados por el frontend. Mapea

además los roles de base de datos (`operario` , `coordinador`) a los roles frontend (`operator` , `coordinator`).

- **`savePuesto(puestoData)`** : Ejecuta un `INSERT ... ON CONFLICT` o actualiza el nombre, sede y orden del puesto, sincronizando sus horarios asociados en una sola operación.
- **`deletePuesto(puestoId)`** : Elimina el puesto y por cascada sus horarios asociados.
- **`saveAsignacion(asg)`** : Realiza un `UPSERT` en `asignaciones_turno` usando `ON CONFLICT(puesto_id, horario_id, fecha_turno) DO UPDATE SET ...` para asignar o reasignar un operario a una celda.
- **`removeAsignacion(puestoId, horarioId, date)`** : Limpia la asignación de una celda para dejarla vacante.
- **`saveEmployee(empData)`** : Inserta o actualiza un colaborador. Transforma el rol de TypeScript al valor del check constraint de SQLite y guarda sus colores visuales.
- **`deleteEmployee(empId)`** : Ejecuta un borrado seguro en 3 pasos:
  1. Desasigna al colaborador de los turnos activos en `asignaciones_turno` .
  2. Elimina sus acumulados de horas de la semana en `acumulados_laborales_2026` .
  3. Si el colaborador tiene registros históricos en la bitácora de novedades (protegidos por `ON DELETE RESTRICT`), realiza un borrado lógico (`UPDATE empleados SET activo = 0`) para cumplir la normativa legal; si no tiene registros históricos, realiza el borrado físico (`DELETE`).
- **`saveBitacoraNovedad(novelty)`** : Inserta el registro inmutable de auditoría con fecha exacta y los JSON de estado previo y nuevo.

4.2

Router HTTP ligero sin dependencias externas:

- **`parseJsonBody(req)`** : Función basada en Streams que concatena los chunks del cuerpo HTTP y los parsea a JSON.
- **`handleApiRequest(req, res, next)`** : Evalúa la URL y el método HTTP para despachar a `sqliteService.js` :
  - `GET /api/status` : Diagnóstico y salud de la base de datos.
  - `GET /api/initial-data` : Carga inicial de toda la matriz.
  - `POST / PUT / DELETE /api/puestos` : Mutaciones sobre puestos y horarios.
  - `POST / DELETE /api/asignaciones` : Asignación de turnos.
  - `POST / PUT / DELETE /api/empleados` : Altas, bajas y cambios de colaboradores.
  - `POST /api/bitacora` : Registro de novedades de auditoría.

4.3

Servidor HTTP autónomo para entornos de producción o staging:

- Configura cabeceras CORS (`Access-Control-Allow-Origin: *`).
- Intercepta rutas `/api/*` derivándolas a `handleApiRequest` .
- Sirve estáticos de la carpeta `dist/` con soporte para SPA (Single Page Application fallback hacia `dist/index.html`).
- Escucha por defecto en el puerto `3001` .

4.4

Middleware integrado en el servidor de desarrollo Vite:

- Define el plugin `sqliteApiPlugin()` .
- Intercepta cualquier llamada iniciada con `/api/` en `http://localhost:5173` y la resuelve directamente en memoria con `handleApiRequest` de `server/apiRouter.js` .
- **Beneficio clave:** El desarrollador solo corre `npm run dev` y tiene frontend y base de datos SQLite funcionando en el mismo puerto sin requerir proxies externos.

5. Lógica de Negocio y Motores Normativos (`src/utils/` y `src/services/`)

5.1

Implementa las exigencias normativas laborales colombianas de 2026 (RF-01 y RF-02):

- **`COLOMBIAN_LABOR_CONFIG_2026`** :
  - `WEEKLY_LEGAL_LIMIT_HOURS: 42` : Tope máximo semanal según la reducción progresiva de la Ley 2101 de 2021.
  - `NIGHT_START_HOUR: 19` (07:00 PM) y `NIGHT_END_HOUR: 6` (06:00 AM): Franja nocturna ampliada de 11 horas.
  - Porcentajes de recargo: Recargo nocturno (+35%), Horas extras diurnas (+25%), Horas extras nocturnas (+75%), Dominicales y festivos (+75% / +100%).
- **`COLOMBIAN_HOLIDAYS_2026`** : Conjunto (`Set`) con los 18 festivos oficiales colombianos del año 2026 calculados conforme a la Ley Emiliani (Ley 51 de 1983).
- **`calculateShiftMinutes(startTime, endTime)`** : Calcula con precisión de minutos la duración del turno dividiéndolo en:
  - Minutos en franja diurna (06:00 a 19:00).

    - Minutos en franja nocturna (19:00 a 06:00, con soporte para turnos que cruzan la medianoche como `14:00 - 00:00` o `22:00 - 06:00`).

- `calculateShiftLabor(shift, date)`: Clasifica las horas del turno en horas ordinarias diurnas, recargo nocturno, o dominical/festivo si el día cae en domingo o en la lista de festivos.
- `calculateWeeklyLaborAccumulated(employeeId, weekDates, schedules)`:
    - Acumula el total de horas trabajadas por el operario de lunes a domingo.
    - Si la suma supera 42 horas, activa `isOverLimit: true`, fija las horas ordinarias en 42h y clasifica el excedente como horas extras (`overLimitHours`).
    - Genera el porcentaje de capacidad para pintar las barras de progreso estilo Everhour en la columna izquierda de la malla.

5.2

Generador de archivos de nómina descargables en el navegador (RF-04):

- `exportPayrollToExcel(...)`: Utiliza la librería `xlsx` (SheetJS) para generar un archivo binario `.xlsx` con columnas estructuradas (Cédula, Empleado, Cargo, Sede, Total Horas, Horas Ordinarias, Recargo Nocturno 19-06, Extras Diurnas, Extras Nocturnas, Dominicales y Estado Tope 42h).
- `exportPayrollToCSV(...)`: Genera un archivo `.csv` con codificación UTF-8 con BOM (`\uFEFF`) para compatibilidad nativa con Microsoft Excel en español (delimitado por punto y coma `;`).
- `exportPayrollToTXT(...)`: Genera un archivo de texto plano estructurado con anchos de columna fijos, compatible con software contable tradicional (SAP, Novasoft, Helisa, Siigo).

5.3

Conector con flujos de n8n y WhatsApp (RF-05):

- `maskPhoneNumber(phone)` y `maskEmail(email)`: Funciones de enmascaramiento para dar cumplimiento a la Ley 1581 de 2012 de Habeas Data y protección de datos sensibles (RNF-02).
- `dispatchScheduleWebhook(payload)` **(Flujo 6)**: Envía por HTTP POST la malla consolidada al webhook `VITE_N8N_WEBHOOK_SEND_SCHEDULE` para su distribución por WhatsApp a los operarios.
- `dispatchHighPriorityAlertWebhook(alert)` **(Flujo 7)**: Se dispara automáticamente cuando un cambio de turno afecta el día en curso o el día siguiente (`priority: 'HIGH'`).
- `retryNotificationLog(logId)`: Sistema de reintentos manuales ante fallos de entrega o caídas de red.

5.4

Cliente HTTP del frontend:

- Métodos tipados `checkDbConnection()`, `fetchInitialDataFromDb()`, `persistPuestoInDb()`, `deletePuestoInDb()`, `persistEmployeeInDb()`, `deleteEmployeeInDb()`, `persistAsignacionInDb()`, `persistBitacoraInDb()`.
- Envía la cabecera `X-ZonaP-Application: NexaFlow-Frontend`.

6. Estado Global React (`src/context/AppContext.tsx`)

Es el núcleo central de reactividad de toda la aplicación. Provee:

1. **Conexión Automática a SQLite al Iniciar**:
   - En el `useEffect` de montaje, ejecuta `checkDbConnection()` y `fetchInitialDataFromDb()`.
   - Si la base de datos responde, inicializa los estados de empleados, puestos, asignaciones y bitácora desde SQLite y dispara un Toast verde de confirmación.
2. **Sincronización Bidireccional de Matriz**:
   - Convierte automáticamente la matriz bidimensional de celdas (`puestoAssignments`) al modelo de horarios diarios (`DaySchedule`) para que el motor laboral `colombianLaborEngine` calcule en tiempo real las horas de todos los empleados.
3. **Gestión de Puestos y Horarios**:
   - `addPuesto`, `updatePuesto`, `deletePuesto`, `addHorarioToPuesto`, `updateHorario`, `deleteHorario`.
   - Actualizan el estado local de React y guardan inmediatamente en SQLite.
4. **Gestión de Empleados (Altas, Bajas y Edición)**:
   - `addEmployee`: Genera ID único, crea el operario con sus colores y lo persiste en `database/zonap.db`.
   - `updateEmployee`: Modifica los datos del colaborador en memoria y en SQLite.
   - `deleteEmployee`: Remueve al empleado, libera todos sus turnos en la grilla y actualiza SQLite de forma segura.
5. **Asignación de Celdas y Novedades Operativas (RF-03)**:
   - `assignEmployeeToPuesto`: Cuando el coordinador reasigna una celda, detecta si hubo cambio de operario o motivo de novedad.

- Si hubo novedad, crea un registro inmutable en `auditLogs`, lo guarda en la tabla `bitacora_novedades` de SQLite y dispara el webhook de prioridad alta si afecta hoy o mañana.

6. **Sistema de Toasts**: Notificaciones flotantes no bloqueantes con autodestrucción en 4.5 segundos.

7. Módulos y Componentes de Interfaz (`src/components/`)

7.1

Barra superior de control global:

- **Branding Zona P**: Identidad visual con badge de cumplimiento de la norma laboral 2026.
- **Navegador Semanal (Semana de Estudio 26/10/2026 - 01/11/2026)**: Flechas de semana anterior, semana siguiente y botón de semana en curso.
- **Filtro Rápido de Sede**: Botones para filtrar instantáneamente por `Todas`, `S1 Centro`, `S2 Norte`, `S3 Laureles`.
- **Simulador de Rol**: Selector desplegable que permite alternar la vista entre:
  - `Administrador`: Control total.
  - `Coordinador de Sede`: Edición operativa y novedades.
  - `Operario`: Modo solo lectura personal.
  - `Auditor RRHH`: Fiscalización legal y nómina.
- **Badge de SQLite**: Muestra el indicador 🟢 `SQLite: Conectado / database/zonap.db`.

7.2

Barra lateral con los 5 módulos del sistema:

1. **Malla de Turnos** (Resource Planner con badge de alertas rojas si alguien supera las 42 horas).
2. **Gestión de Empleados** (Altas, bajas, roles y directorio con contador en tiempo real).
3. **Reportes de Nómina** (Descargas Excel, CSV, TXT - RF-04).
4. **Bitácora Novedades** (Changelog inmutable - RF-03).
5. **Legal & Backups** (Norma 2026, webhooks y retención de 90 días - RF-06).

7.3

La vista central inspirada en la grilla física de Zona P y el diseño de Everhour:

- **Columna de Puestos (Izquierda)**:
    - Agrupada por sedes.
    - Puestos editables en línea haciendo clic sobre el nombre (para que el coordinador pueda llamarlos como desee).
    - Botones para añadir puestos (`+ Nuevo Puesto`) y configurar horarios.
- **Columna de Horarios**: Muestra franjas horarias como `06:00 - 14:00`, `14:00 - 22:00`, etc.
- **Columnas de Días (Lunes a Domingo)**:
    - Celdas operativas interactivas.
    - Si el día es inactivo para ese horario, se muestra en gris claro neutral `#C0C0C0`.
    - Si está asignada, dibuja la píldora con el nombre del operario en su color pastel/vibrante asignado.
    - Si fue cubierto por novedad, muestra borde magenta distintivo.
    - Permite hacer clic en cualquier celda para abrir el modal de asignación y novedad.
- **Selector de Modo de Visualización**: Permite alternar entre la vista matricial por puestos y la vista por operarios con barras de capacidad de horas (`XXh / 42h`).

7.4

Modal para asignar un operario a un puesto y día:

- Buscador rápido de operarios disponibles.
- Campo de nota especial (ej. `"08-15"`, `"Relevo almuerzo"`).
- Selector obligatorio de novedad (RF-03) si se está reemplazando a un operario existente.

7.5

Modal para crear o editar puestos:

- Permite al coordinador nombrar el puesto (ej. `"Entrada Vehicular # 2"`, `"Puesto Valet"`).
- Asignar la sede correspondiente.
- Agregar múltiples franjas horarias con selectores de inicio, fin y días de la semana operativos.
- Persiste los cambios directamente en SQLite.

7.6

Módulo completo de administración de personal:

- **Tarjetas KPI**: Conteo de operarios en nómina y desglose por cada una de las 3 sedes.

- **Buscador y Filtros**: Búsqueda instantánea por nombre, documento de identidad, cargo y sede.
- **Tabla Detallada**:
    - Iniciales del operario con su color asignado.
    - Cédula, teléfono para WhatsApp y correo.
    - Previsualización real de la píldora como se ve en la Malla de Turnos.
    - Barra de capacidad semanal con indicador del límite legal de 42 horas.
- **Modal de Alta / Edición**:
    - Formulario completo con validación.
    - **Selector de Paleta de Color de Turno**: 10 presets inspirados en Zona P con previsualización en vivo.
- **Modal de Eliminación Segura**: Advierte cuántos turnos activos tiene el colaborador y desvincula las celdas en SQLite sin romper registros históricos de auditoría.

7.7

Módulo de reportes para liquidación de nómina (RF-04):

- Filtros por rango de fechas, sede y empleado.
- Tabla resumen con desglose de: Horas Ordinarias, Recargos Nocturnos (19:00 a 06:00), Extras Diurnas, Extras Nocturnas, Dominicales/Festivos y Alerta de Exceso de 42h.
- Botones de descarga real de archivos en formatos **Excel (.xlsx)**, **CSV (.csv)** y **TXT (.txt)**.

7.8

Bitácora inmutable de auditoría (RF-03):

- Muestra el registro histórico de todas las modificaciones y novedades de turnos.
- Tarjetas con diff visual (estado previo vs. estado nuevo).
- Motivo tipificado de la novedad, coordinador responsable y fecha exacta.
- Badge de "Prioridad Alta" si afectó turnos de hoy o mañana con indicación de despacho a n8n/WhatsApp.

7.9

Panel administrativo legal y de respaldos (RF-06):

- Visualización de la política de retención de copias de seguridad de 90 días.
- Historial de instantáneas automáticas y manuales con checksum SHA-256 simulado.

- Opción de restaurar el estado del sistema a un snapshot histórico.
- Configuración de URLs de webhooks de n8n para WhatsApp y alertas.

8. Flujo de Datos Típico: Asignar un Turno en la Malla

Para entender cómo interactúan todos los componentes entre sí, observemos el recorrido de una acción:

1. El Coordinador hace clic en una celda en .
2. Se abre . El coordinador selecciona a "Yoiner Beltran" y el motivo "Fuerza mayor".
3. Al hacer clic en "Confirmar", se llama a `assignEmployeeToPuesto` en .
4. `AppContext` actualiza su estado local en memoria (`puestoAssignments`).
5. Inmediatamente se invoca `persistAsignacionInDb` en , que realiza un `POST /api/asignaciones` .
6. El middleware en intercepta la petición y la envía a `handleApiRequest` en .
7. ejecuta el SQL `INSERT ... ON CONFLICT DO UPDATE` contra .
8. Simultáneamente, el motor laboral recalcula las horas semanales del operario; si pasa de 42h, marca alerta roja.
9. Se crea un registro en la tabla `bitacora_novedades` de SQLite.
10. Si el turno es para hoy o mañana, despacha la alerta de prioridad alta al webhook de WhatsApp.
11. Se muestra un Toast en la interfaz informando al usuario que la asignación se guardó con éxito en SQLite.

9. Comandos de Operación

Comando	Acción
`npm run dev`	Inicia la aplicación React con el backend de SQLite integrado en `http://localhost:5173`.
`npm run build`	Compila el código TypeScript (`tsc -b`) y genera los archivos optimizados con Vite en `dist/`.
`npm run db:init`	Ejecuta `database/init_sqlite.js` para recargar los datos semilla de fábrica en `database/zonap.db`.
`npm run server`	Inicia el servidor Node.js independiente en el puerto `3001` sirviendo la API y la app de producción.